題目：仿刻一個「單位換算器」。

規格：
l視窗 480×280，BorderLayout
lNORTH：JComboBox 選換算類型（長度／重量／溫度）
lCENTER：兩列，每列一個 JTextField ＋ 一個 JComboBox（來源單位、目標單位）
l切換換算類型時，兩個單位 JComboBox 的選項要跟著換
 l長度：公尺、公分、英吋、英尺
l重量：公斤、公克、磅、盎司
l溫度：攝氏、華氏、克氏
lSOUTH：「換算」按鈕，結果顯示在第二個 JTextField

這題可以用 Java Swing 的 `JFrame`、`JPanel`、`JComboBox` 和 `JTextField` 完成。我會按照題目要求使用 `BorderLayout`，並讓切換「長度／重量／溫度」時，來源單位和目標單位的選項自動更新。

先整理版面配置：

- NORTH：換算類型下拉選單
- CENTER：兩列輸入區，分別放數值輸入框與單位下拉選單
- SOUTH：換算按鈕
- 換算結果：顯示在第二個 `JTextField`

另外，溫度換算需要使用不同公式，不能像長度和重量一樣只做倍率相乘。

完整 Java 程式碼

檔名請存成 `UnitConverter.java`，因為公開類別名稱是 `UnitConverter`。

```
import javax.swing.*;
import java.awt.*;

public class UnitConverter extends JFrame {

    private JComboBox<String> typeBox;
    private JComboBox<String> fromBox;
    private JComboBox<String> toBox;

    private JTextField inputField;
    private JTextField resultField;

    private JButton convertButton;

    // 長度、重量、溫度的單位
    private String[][] units = {
        {"公尺", "公分", "英吋", "英尺"},
```

```
        {"公斤", "公克", "磅", "盎司"},
        {"攝氏", "華氏", "克氏"}
    };

    public UnitConverter() {

        setTitle("單位換算器");
        setSize(480, 280);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(10, 10));

        // NORTH：換算類型
        typeBox = new JComboBox<>(
            new String[]{"長度", "重量", "溫度"}
        );

        JPanel northPanel = new JPanel();
        northPanel.add(new JLabel("換算類型："));
        northPanel.add(typeBox);

        add(northPanel, BorderLayout.NORTH);

        // CENTER：兩列輸入區
        inputField = new JTextField(10);
        resultField = new JTextField(10);
        resultField.setEditable(false);

        fromBox = new JComboBox<>(units[0]);
        toBox = new JComboBox<>(units[0]);

        JPanel centerPanel = new JPanel(
            new GridLayout(2, 1, 5, 10)
        );

        JPanel row1 = new JPanel();
        row1.add(new JLabel("來源數值："));
        row1.add(inputField);
        row1.add(fromBox);

        JPanel row2 = new JPanel();
        row2.add(new JLabel("換算結果："));
        row2.add(resultField);
        row2.add(toBox);

        centerPanel.add(row1);
        centerPanel.add(row2);
```

```
    add(centerPanel, BorderLayout.CENTER);

    // SOUTH：換算按鈕
    convertButton = new JButton("換算");

    JPanel southPanel = new JPanel();
    southPanel.add(convertButton);

    add(southPanel, BorderLayout.SOUTH);

    // 切換類型時，更新兩個單位選單
    typeBox.addActionListener(e -> {
        int index = typeBox.getSelectedIndex();

        fromBox.setModel(
            new DefaultComboBoxModel<>(units[index])
        );

        toBox.setModel(
            new DefaultComboBoxModel<>(units[index])
        );

        resultField.setText("");
    });

    // 按下換算按鈕
    convertButton.addActionListener(e -> convert());

    setVisible(true);
}

// 執行換算
private void convert() {

    try {
        double value = Double.parseDouble(
            inputField.getText().trim()
        );

        int type = typeBox.getSelectedIndex();
        int from = fromBox.getSelectedIndex();
        int to = toBox.getSelectedIndex();

        double result;

        if (type == 0) {
            // 長度：先換成公尺，再換成目標單位
            double[] factor = {
```

```
                1.0, 0.01, 0.0254, 0.3048
            };

            result = value * factor[from] / factor[to];

        } else if (type == 1) {
            // 重量：先換成公斤，再換成目標單位
            double[] factor = {
                1.0, 0.001, 0.45359237, 0.028349523125
            };

            result = value * factor[from] / factor[to];

        } else {
            // 溫度：先換成攝氏，再換成目標單位
            double celsius;

            if (from == 0) {
                celsius = value;
            } else if (from == 1) {
                celsius = (value - 32) * 5.0 / 9.0;
            } else {
                celsius = value - 273.15;
            }

            if (to == 0) {
                result = celsius;
            } else if (to == 1) {
                result = celsius * 9.0 / 5.0 + 32;
            } else {
                result = celsius + 273.15;
            }
        }

        resultField.setText(
            String.format("%.4f", result)
        );

    } catch (NumberFormatException ex) {
        JOptionPane.showMessageDialog(
            this,
            "請輸入有效的數字！",
            "輸入錯誤",
            JOptionPane.ERROR_MESSAGE
        );
    }
}
```

```
    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            new UnitConverter();
        });
    }
}
```

程式執行後的操作方式

範例一：長度換算

輸入 100，來源選「公分」，目標選「公尺」，按下換算，結果為 1.0000。

範例二：重量換算

輸入 1，來源選「公斤」，目標選「公克」，結果為 1000.0000。

範例三：溫度換算

輸入 100，來源選「攝氏」，目標選「華氏」，結果為 212.0000。

老師規格與程式碼的對應

題目要求	程式碼實作
視窗 480 × 280	`setSize(480, 280)`
使用 `BorderLayout`	`setLayout(new BorderLayout(10, 10))`
NORTH 選換算類型	`typeBox`
CENTER 兩列輸入區	`GridLayout(2, 1, 5, 10)`
每列有輸入框與單位選單	`inputField`、`fromBox`、`resultField`、`toBox`

切換類型更新單位	`typeBox.addActionListener(...)`
SOUTH 換算按鈕	`convertButton`
結果顯示在第二個輸入框	`resultField.setText(...)`

注意兩個實作細節：

1. 第二個 `JTextField` 設定了 `setEditable(false)`，因此它只顯示結果，不會被使用者直接修改。
2. 題目中的「克氏」在此程式中視為 Kelvin（開爾文溫標），與攝氏溫標的換算使用 $K = {}^{\circ}C + 273.15$。

執行方式：

javac UnitConverter.java
java UnitConverter

這份程式已涵蓋三種換算、單位選單動態切換、數字輸入錯誤處理，以及指定的視窗版面配置。

要怎麼確保單位兩個會一起變換呢

要確保兩個單位下拉選單（來源單位、目標單位）會一起變換，關鍵是：在換算類型的 `JComboBox` 加上 `addActionListener()`，並且在同一個事件中，同時更新 `fromBox` 和 `toBox`。

你可以直接在 `UnitConverter` 建構子裡加入或確認有以下程式碼：

```
typeBox.addActionListener(e -> {
    int index = typeBox.getSelectedIndex();

    // 同時更新來源單位
    fromBox.setModel(
        new DefaultComboBoxModel<>(units[index])
    );

    // 同時更新目標單位
    toBox.setModel(
        new DefaultComboBoxModel<>(units[index])
    );

    // 清除舊的換算結果
```

```
    resultField.setText("");
});
```

為什麼這樣就可以？

假設你的 `units` 是這樣設定的：

```
String[][] units = {
    {"公尺", "公分", "英吋", "英尺"},
    {"公斤", "公克", "磅", "盎司"},
    {"攝氏", "華氏", "克氏"}
};
```

`typeBox.getSelectedIndex()` 會取得目前選擇的類型編號：

換算類型	index	來源、目標單位選項
長度	0	公尺、公分、英吋、英尺
重量	1	公斤、公克、磅、盎司
溫度	2	攝氏、華氏、克氏

例如使用者把換算類型從「長度」切換成「重量」：

1. `getSelectedIndex()` 取得 `1`。
2. `units[1]` 取得公斤、公克、磅、盎司。
3. `fromBox.setModel(...)` 更新來源單位。
4. `toBox.setModel(...)` 更新目標單位。

因為這兩個更新都寫在同一個事件裡，所以使用者切換類型時，兩個下拉選單就會一起更新。

如果切換後還是沒有變換，檢查這 3 件事

- `fromBox` 和 `toBox` 必須先建立，再設定事件。
- 更新時必須使用 `units[index]`，不能固定寫成 `units[0]`。
- 兩個 `setModel()` 都要執行，只更新其中一個，另一個不會跟著改變。

你可以依序測試「長度 → 重量 → 溫度」，確認兩個下拉選單的選項都會同步改變。

解釋完整程式碼

我會按照你前面那份 `UnitConverter.java`，從最上面的 `import` 開始，一路解釋到最後的 `main()`，包含每個變數的用途、視窗怎麼排版、兩個單位下拉選單怎麼同步，以及長度、重量、溫度的換算公式。

你可以把整個程式想成 3 個部分：

1. 建立介面：把視窗、輸入框、下拉選單和按鈕放好。
2. 監聽操作：使用者切換換算類型或按下換算按鈕時，程式就執行相應動作。
3. 計算結果：根據選擇的單位計算，並把結果顯示在第二個輸入框。

一、匯入套件與建立類別

```
import javax.swing.*;

import java.awt.*;


public class UnitConverter extends JFrame {
```

1. `import javax.swing.*;`

匯入 Java Swing 的圖形介面元件。

這份程式會用到：

元件	用途
`JFrame`	建立主視窗
`JPanel`	把元件分組
`JLabel`	顯示文字標籤
`JTextField`	輸入數字、顯示結果

`JComboBox`	建立下拉選單
`JButton`	建立換算按鈕
`JOptionPane`	顯示錯誤提示視窗

2. `import java.awt.*;`

匯入 Java AWT 的版面配置工具，這裡主要使用：

- `BorderLayout`：把視窗分成北、南、東、西、中間等區域。
- `GridLayout`：把面板切成固定列數與欄數。

3. `public class UnitConverter extends JFrame`

這行代表建立一個名為 `UnitConverter` 的類別。

`extends JFrame` 表示這個類別繼承 `JFrame`，所以 `UnitConverter` 本身就是一個視窗，可以直接使用 `setTitle()`、`setSize()`、`add()` 等方法。

二、宣告介面元件

```
private JComboBox<String> typeBox;

private JComboBox<String> fromBox;

private JComboBox<String> toBox;


private JTextField inputField;

private JTextField resultField;


private JButton convertButton;
```

這些是物件的變數宣告，讓程式其他部分也能操作這些元件。

變數	意義
`typeBox`	選擇長度、重量或溫度
`fromBox`	選擇來源單位
`toBox`	選擇目標單位
`inputField`	輸入原始數值
`resultField`	顯示換算結果
`convertButton`	執行換算的按鈕

例如：

private JTextField inputField;

這行只是宣告變數，還沒有真正建立輸入框。後面執行：

inputField = new JTextField(10);

才會建立一個欄位寬度約為 10 個字元的文字輸入框。

`private` 表示這些變數只能直接從 `UnitConverter` 類別內存取，能避免其他類別隨意修改它們。

`JComboBox<String>` 中的 `String` 表示下拉選單的選項是文字，例如「公尺」、「公斤」。

三、設定三種單位

private String[][] units = {

{"公尺", "公分", "英吋", "英尺"},

```
    {"公斤", "公克", "磅", "盎司"},
    {"攝氏", "華氏", "克氏"}
};
```

這是一個二維字串陣列，用來集中保存三種換算類型的單位。

可以把它想成三份不同的清單：

陣列索引	換算類型	`units[index]`
0	長度	公尺、公分、英吋、英尺
1	重量	公斤、公克、磅、盎司
2	溫度	攝氏、華氏、克氏

例如：

units[0]

取得長度的所有單位。

units[1]

取得重量的所有單位。

units[2]

取得溫度的所有單位。

這樣設計的好處是，之後要更新下拉選單時，不必重複寫三組不同的程式碼，只要取得正確的索引就可以了。

四、建立視窗與設定版面

```
public UnitConverter() {

    setTitle("單位換算器");
```

```
    setSize(480, 280);

    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

    setLocationRelativeTo(null);

    setLayout(new BorderLayout(10, 10));
```

這一段是建構子，也就是建立 `UnitConverter` 物件時會執行的程式。

每一行的作用

- `setTitle("單位換算器")`：設定視窗標題。
- `setSize(480, 280)`：設定視窗大小為寬 480、高 280。
- `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`：關閉視窗時，結束程式。
- `setLocationRelativeTo(null)`：讓視窗出現在螢幕中央。
- `setLayout(new BorderLayout(10, 10))`：設定主視窗的版面配置，區域之間的水平、垂直間距各為 10 像素。

`BorderLayout` 是什麼？

NORTH：上方

換算類型下拉選單

CENTER：中間

第一列：來源數值＋來源單位

第二列：換算結果＋目標單位

SOUTH：下方

換算按鈕

`BorderLayout` 本身不會自動把所有元件平均排成兩列，因此程式另外建立 `JPanel`，再用 `GridLayout` 排列中間的元件。

五、建立 NORTH 區域：換算類型

```
typeBox = new JComboBox<>(

    new String[]{"長度", "重量", "溫度"}

);
```

```
JPanel northPanel = new JPanel();
northPanel.add(new JLabel("換算類型："));
northPanel.add(typeBox);


add(northPanel, BorderLayout.NORTH);
```

這段負責建立視窗上方的換算類型選單。

逐行解釋

第 1 步：建立下拉選單

```
typeBox = new JComboBox<>(
    new String[]{"長度", "重量", "溫度"}
);
```

`new JComboBox<>()` 會建立一個下拉選單，並放入三個選項。

第 2 步：建立面板

```
JPanel northPanel = new JPanel();
```

`JPanel` 可以把相關元件放在一起，方便整體安排位置。

第 3 步：把文字與選單放進面板

```
northPanel.add(new JLabel("換算類型："));
northPanel.add(typeBox);
```

畫面上就會出現「換算類型：」和下拉選單。

第 4 步：把面板放到視窗上方

```
add(northPanel, BorderLayout.NORTH);
```

這裡的 `add()` 是將面板加入主視窗，`BorderLayout.NORTH` 指定放在北側，也就是上方。

注意：`northPanel.add()` 和 `add(northPanel, BorderLayout.NORTH)` 的差別在於，前者把元件放進面板，後者把整個面板放進視窗。

六、建立 CENTER 區域：兩列輸入區

1. 建立輸入框和單位選單

```
inputField = new JTextField(10);

resultField = new JTextField(10);

resultField.setEditable(false);


fromBox = new JComboBox<>(units[0]);

toBox = new JComboBox<>(units[0]);
```

這裡會建立四個主要元件。

程式碼	功能
`new JTextField(10)`	建立約 10 個字元寬的文字框
`resultField.setEditable(false)`	讓結果欄位只能顯示，不能直接編輯
`new JComboBox<>(units[0])`	建立單位選單，初始選項為長度單位

為什麼 `fromBox` 和 `toBox` 都使用 `units[0]`？

因為程式一開始預設選擇「長度」，所以兩個單位選單都先顯示公尺、公分、英吋、英尺。

之後切換類型時，再把它們更新成重量或溫度的單位。

2. 建立兩列的版面

```
JPanel centerPanel = new JPanel(
    new GridLayout(2, 1, 5, 10)
);
```

`GridLayout(2, 1, 5, 10)` 的意思是：

- `2`：兩列。
- `1`：一欄。

- `5`：欄與欄之間的水平間距。
- `10`：列與列之間的垂直間距。

因此，`centerPanel` 會被切成上下兩列，每一列各放一個子面板。

3. 建立第一列：輸入來源數值

```
JPanel row1 = new JPanel();

row1.add(new JLabel("來源數值："));

row1.add(inputField);

row1.add(fromBox);
```

第一列依序加入：

1. 「來源數值：」標籤。
2. 原始數值輸入框。
3. 來源單位選單。

例如，使用者輸入 `100`，來源單位選「公分」。

4. 建立第二列：顯示換算結果

```
JPanel row2 = new JPanel();

row2.add(new JLabel("換算結果："));

row2.add(resultField);

row2.add(toBox);
```

第二列依序加入：

1. 「換算結果：」標籤。
2. 結果輸出框。
3. 目標單位選單。

例如，目標單位選「公尺」，按下換算後，第二個文字框就會顯示 `1.0000`。

5. 將兩列加入 CENTER

```
centerPanel.add(row1);

centerPanel.add(row2);



add(centerPanel, BorderLayout.CENTER);
```

先把 row1、row2 加進 centerPanel，再把 centerPanel 放到主視窗中央。

這裡有三層關係：

主視窗 JFrame

centerPanel：兩列一欄

row1：來源數值、輸入框、來源單位

row2：換算結果、結果框、目標單位

這就是為什麼程式要先建立 row1 和 row2，而不是把所有元件直接加入主視窗。

七、建立 SOUTH 區域：換算按鈕

convertButton = new JButton("換算");

JPanel southPanel = new JPanel();

southPanel.add(convertButton);

add(southPanel, BorderLayout.SOUTH);

這段的邏輯與 NORTH 很像：

1. new JButton("換算")：建立按鈕。
2. new JPanel()：建立面板。
3. southPanel.add(convertButton)：把按鈕放入面板。
4. add(southPanel, BorderLayout.SOUTH)：把面板放到視窗下方。

這裡只是建立按鈕和安排位置，還沒有定義按鈕的功能。

按鈕的功能會在後面透過 addActionListener() 設定。

八、最重要的部分：切換類型時，同步更新兩個單位選單

typeBox.addActionListener(e -> {

int index = typeBox.getSelectedIndex();

fromBox.setModel(

```
        new DefaultComboBoxModel<>(units[index])
    );

    toBox.setModel(
        new DefaultComboBoxModel<>(units[index])
    );

    resultField.setText("");
});
```

這段是整個程式中負責「切換類型時，兩個下拉選單一起變換」的關鍵。

1. `addActionListener()` 是什麼？

```
typeBox.addActionListener(e -> {
    // 切換選項後要執行的程式
});
```

意思是：當使用者操作 `typeBox`、選擇不同的換算類型時，就執行大括號裡面的程式。

`e -> { ... }` 是 Java 的 Lambda 寫法，可以簡潔地表示事件發生時要執行的動作。

2. 取得使用者選擇的類型

```
int index = typeBox.getSelectedIndex();
```

`getSelectedIndex()` 會回傳目前選中項目的索引，從 `0` 開始計算。

例如：

- 選「長度」→ `index = 0`
- 選「重量」→ `index = 1`
- 選「溫度」→ `index = 2`

這個 `index` 就是用來選擇正確單位清單的依據。

3. 更新來源單位

```
fromBox.setModel(
```

new DefaultComboBoxModel<>(units[index])

);

這行會建立一個新的下拉選單資料模型，並將 `units[index]` 設為來源單位的選項。

例如 `index = 1` 時：

units[1]

取得：

{"公斤", "公克", "磅", "盎司"}

因此，來源單位選單就會更新成這四個選項。

4. 更新目標單位

toBox.setModel(

new DefaultComboBoxModel<>(units[index])

);

這行和上一行的做法完全相同，只是這次更新的是目標單位選單。

為什麼兩個選單會一起更新？

因為兩行都使用同一個 `index`，而且都在同一個事件中執行。

當使用者選擇「重量」時，兩行都會使用 `units[1]`；選擇「溫度」時，兩行都會使用 `units[2]`。

5. 清除舊的換算結果

resultField.setText("");

當換算類型改變時，清空第二個文字框，避免畫面仍然顯示上一種單位的結果。

例如，剛才算出 100 公分等於 1 公尺，接著切換到重量，就不應該繼續顯示舊的長度換算結果。

完整操作流程

使用者切換 `typeBox`

`getSelectedIndex()` 取得索引

更新 `fromBox`

來源單位

更新 `toBox`

目標單位

清空舊的換算結果

九、設定換算按鈕的事件

convertButton.addActionListener(e -> convert());

這行的意思是：當使用者按下「換算」按鈕時，就呼叫 `convert()` 方法。

這裡的 `convert()` 是我們自己定義的方法，不是 Java 內建的方法。

你可以把它理解成：

- 按下按鈕 → 觸發事件。
- 事件發生 → 執行 `convert()`。
- `convert()` → 讀取數值與單位，計算結果，顯示在第二個文字框。

真正的計算邏輯就在下一段。

十、`convert()`：讀取輸入資料

```
private void convert() {


    try {
        double value = Double.parseDouble(
            inputField.getText().trim()
        );


        int type = typeBox.getSelectedIndex();
        int from = fromBox.getSelectedIndex();
        int to = toBox.getSelectedIndex();


        double result;
```

1. `private void convert()`

這是自訂方法的宣告。

- `private`:只能從這個類別內直接呼叫。
- `void`:方法不會直接回傳計算結果。
- `convert()`:方法名稱，可以自行命名。

計算結果會直接顯示在 `resultField`，因此不需要用 `return` 回傳數值。

2. 取得使用者輸入的數字

```
double value = Double.parseDouble(
    inputField.getText().trim()
);
```

這行可以拆成三個步驟：

程式碼	功能
`inputField.getText()`	取得文字框裡的內容
`.trim()`	移除文字前後的空白
`Double.parseDouble(...)`	將文字轉成 `double` 數字

例如使用者輸入 `100`：

inputField.getText()

取得的是字串 `"100"`，而不是數字 `100`。

經過 `Double.parseDouble()` 後，才會得到可以參與數學運算的數值 `100.0`。

為什麼使用 `double`？

因為換算可能產生小數，例如 1 英吋等於 2.54 公分，所以不能只使用整數 `int`。

3. 取得選擇的類型與單位

int type = typeBox.getSelectedIndex();

int from = fromBox.getSelectedIndex();

int to = toBox.getSelectedIndex();

這三個變數都是用來記錄下拉選單目前選中的索引。

假設使用者設定如下：

- 換算類型：長度
- 來源單位：公分
- 目標單位：公尺

那麼：

type = 0;

from = 1;

to = 0;

因為長度是第一個選項（索引 `0`），公分是長度清單中的第二個選項（索引 `1`），公尺則是第一個選項（索引 `0`）。

4. 宣告計算結果

double result;

這是用來存放最後換算結果的變數。

注意：這時候還沒有計算，只是先宣告變數，稍後才會依照換算類型賦值。

十一、長度換算：`type == 0`

```
if (type == 0) {


    double[] factor = {

        1.0, 0.01, 0.0254, 0.3048

    };


    result = value * factor[from] / factor[to];
```

}

當 `type == 0` 時，代表使用者選擇了「長度」。

1. 先理解 `factor` 陣列

```
double[] factor = {
    1.0, 0.01, 0.0254, 0.3048
};
```

這個陣列記錄每個單位相當於多少公尺。

索引	單位	換算成公尺
0	公尺	1
1	公分	0.01
2	英吋	0.0254
3	英尺	0.3048

例如：

- 1 公尺 = 1 公尺
- 1 公分 = 0.01 公尺
- 1 英吋 = 0.0254 公尺
- 1 英尺 = 0.3048 公尺

2. 為什麼公式是乘上來源倍率，再除以目標倍率？

```
result = value * factor[from] / factor[to];
```

因為程式採用統一的換算方式：

先把來源數值換成公尺，再從公尺換成目標單位。

假設輸入 100 公分，目標單位為公尺：

value = 100;

factor[from] = 0.01;

factor[to] = 1.0;

代入公式：

$$100 \times 0.01 \div 1.0 = 1$$

所以結果為 1 公尺。

再看另一個例子：1 英尺換算成英吋。

$$1 \times 0.3048 \div 0.0254 = 12$$

所以 1 英尺等於 12 英吋。

這種寫法的優點是，不需要為每一組單位都另外寫一條換算公式。

十二、重量換算：`type == 1`

```
else if (type == 1) {


    double[] factor = {
        1.0, 0.001, 0.45359237, 0.028349523125
    };


    result = value * factor[from] / factor[to];
}
```

這段和長度換算的邏輯相同，只是把共同基準單位改成公斤。

1. 重量倍率表

索引	單位	換算成公斤

0	公斤	1
1	公克	0.001
2	磅	0.45359237
3	盎司	0.028349523125

例如，1 公斤換算成公克：

$$1 \times 1 \div 0.001 = 1000$$

所以結果為 1000 公克。

2. 為什麼不用重新設計一套公式？

因為長度和重量都可以透過倍率換算：

$$\text{結果} = \frac{\text{來源數值} \times \text{來源倍率}}{\text{目標倍率}}$$

只要把 `factor` 陣列換成對應的倍率，便能使用同一種計算邏輯。

這也是程式設計中常見的概念：將共同的計算方式重複利用，只替換不同的資料。

十三、溫度換算：`else`

```
else {
    double celsius;

    if (from == 0) {
        celsius = value;
    } else if (from == 1) {
```

```
        celsius = (value - 32) * 5.0 / 9.0;
    } else {
        celsius = value - 273.15;
    }

    if (to == 0) {
        result = celsius;
    } else if (to == 1) {
        result = celsius * 9.0 / 5.0 + 32;
    } else {
        result = celsius + 273.15;
    }
}
```

溫度換算和長度、重量不同，因為攝氏、華氏與開爾文之間不是單純的倍率關係。

因此程式選擇先統一轉成攝氏，再從攝氏轉成目標溫標。

1. 第一個 `if`：轉成攝氏

`double celsius;`

宣告一個變數，儲存換算後的攝氏溫度。

接著判斷來源單位：

from	來源單位	轉成攝氏的公式
0	攝氏	$C = C$
1	華氏	$C = (F - 32) \times 5/9$

2	克氏／開爾文	$C = K - 273.15$

例如，輸入 100 華氏：

$$C = (100 - 32) \times \frac{5}{9} = 37.7778$$

所以 `celsius` 約為 37.7778。

2. 第二個 `if`：從攝氏轉成目標單位

`to`	目標單位	轉換公式
0	攝氏	$C = C$
1	華氏	$F = C \times 9/5 + 32$
2	克氏／開爾文	$K = C + 273.15$

例如，輸入 100 攝氏，目標單位選華氏：

$$F = 100 \times \frac{9}{5} + 32 = 212$$

最後 `result` 就會得到 `212`。

為什麼這裡不能直接使用長度的 `factor` 公式？

因為華氏和攝氏之間除了倍率，還需要加減 32；開爾文和攝氏之間則需要加減 273.15。若直接用倍率相乘，結果就會錯誤。

十四、把結果顯示在第二個文字框

```
resultField.setText(
    String.format("%.4f", result)
```

);

這行負責把計算結果轉成文字，放進第二個 `JTextField`。

`String.format("%.4f", result)` 是什麼？

`String.format()` 可以依照指定格式，把數值轉換成字串。

其中：

- `%f`：浮點數格式。
- `.4`：小數點後保留 4 位。

例如：

原始結果	顯示內容
1	`1.0000`
2.54	`2.5400`
37.777777	`37.7778`

接著：

resultField.setText(...)

就會將格式化後的文字顯示在第二個文字框。

注意：這裡只是控制顯示的小數位數，不代表實際計算只能保留四位小數。

十五、處理錯誤輸入：`try` 和 `catch`

```
try {
    double value = Double.parseDouble(
        inputField.getText().trim()
    );
```

```
    // 後續換算程式
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(
        this,
        "請輸入有效的數字！",
        "輸入錯誤",
        JOptionPane.ERROR_MESSAGE
    );
}
```

這段的用途是避免使用者輸入非數字內容時，程式直接出現錯誤並停止處理。

例如，使用者輸入：

- `abc`
- `你好`
- 空白內容

這些內容都無法正常轉換成 `double`，因此 `Double.parseDouble()` 會拋出 `NumberFormatException`。

執行流程

1. 程式進入 `try`，嘗試讀取並轉換輸入值。
2. 如果輸入有效，就繼續進行換算。
3. 如果轉換失敗，就跳到 `catch`。
4. `JOptionPane.showMessageDialog()` 顯示「請輸入有效的數字！」。

這裡的 `this` 代表目前的 `UnitConverter` 視窗；`JOptionPane.ERROR_MESSAGE` 則指定使用錯誤提示的圖示。

十六、最後的 `main()`：啟動程式

```
public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        new UnitConverter();
```

```
    });
}
```

這是 Java 程式的進入點。

逐行解釋

`public static void main(String[] args)` 是 Java 應用程式開始執行時會尋找的方法。

new UnitConverter();

這行會建立一個 `UnitConverter` 物件，接著執行建構子中的程式，包括建立視窗、設定版面、建立事件監聽器等。

SwingUtilities.invokeLater(() -> {

    new UnitConverter();

});

這是 Swing 建議的啟動方式，會把建立圖形介面的工作安排到事件派發執行緒（Event Dispatch Thread，EDT）執行，讓介面事件能依照 Swing 的執行緒規則處理。

十七、整份程式的執行流程總整理

1. 執行 `main()`

建立 UnitConverter 視窗

2. 執行建構子

設定 480 × 280、BorderLayout、各區域與元件

3. 使用者選擇換算類型

更新來源與目標單位選單

4. 使用者輸入數值、選擇單位

等待按下換算按鈕

5. 執行 `convert()`

長度

倍率換算

重量

倍率換算

溫度

公式換算

6. 顯示換算結果

透過 `resultField.setText()` 更新第二個文字框

十八、如果老師要求解釋程式，你最需要記住什麼？

核心觀念	一句話記憶
`JFrame`	主視窗
`JPanel`	元件的容器，可以分組排版
`BorderLayout`	決定面板放在視窗哪個區域
`GridLayout`	決定面板內的列數與欄數
`JComboBox`	下拉選單
`addActionListener()`	使用者操作後執行指定程式
`getSelectedIndex()`	取得目前選項的索引
`setModel()`	更新下拉選單的選項資料

`getText()`	取得文字框的內容
`setText()`	設定文字框的內容
`try-catch`	處理可能發生的錯誤
`String.format()`	控制結果顯示格式

最後特別提醒:這份程式最核心的兩個方法是:

- `typeBox.addActionListener()`: 負責讓來源、目標單位選單隨著換算類型同步更新。
- `convert()`: 負責取得輸入資料、判斷換算類型、執行公式並顯示結果。

只要理解這兩段的作用, 你就掌握了這份單位換算器最重要的程式邏輯。